Conflent Kafka的最佳實務做法

NetApp Solutions

NetApp
July 11, 2022

This PDF was generated from https://docs.netapp.com/zh-tw/netapp-solutions/data-analytics/confluent-kafka-introduction.html on July 11, 2022. Always check docs.netapp.com for the latest.

目錄

Conflent Kafka的最佳實務做法 . . . 1
TR-4912：NetApp的Conflent Kafka階層式儲存設備最佳實務準則 . . . 1
解決方案架構詳細資料 . . . 2
技術總覽 . . . 3
一致驗證 . . . 8
可擴充性的效能測試 . . . 11
Connent S3連接器 . . . 13
ConFluent自我平衡叢集 . . . 22
最佳實務準則 . . . 22
規模調整 . . . 23
結論 . . . 26

Conflent Kafka的最佳實務做法

TR-4912：NetApp的Conflent Kafka階層式儲存設備最佳實務準則

Karthikeyan Nagalingam、Joseph Kandatilparambil、NetApp Rankesh Kumar、Conflent

Apache Kafka是一個社群分散式事件串流平台、能夠每天處理數兆次的事件。最初構想為訊息佇列的Kafka是以分散式提交記錄的抽象化為基礎。自從LinkedIn於2011年建立並開放原始碼之後、Kafka便從訊息佇列發展成功能完善的活動串流平台。Conflent提供Apache Kafka的Conflent Platform發佈版本。Conflent Platform為卡夫卡（Kafka）提供額外的社群和商業功能、旨在提升營運者和開發人員大規模上線的串流體驗。

本文件說明在NetApp物件儲存產品上使用ConFluent分層儲存設備的最佳實務準則、提供下列內容：

- 運用NetApp物件儲存設備進行一致驗證–NetApp StorageGRID 產品特色
- 階層式儲存效能測試
- NetApp儲存系統上的Conflent最佳實務準則

為何要使用一致的分層儲存設備？

Connent已成為許多應用程式的預設即時串流平台、尤其是巨量資料、分析和串流工作負載。階層式儲存設備可讓使用者在ConFluent平台中、將運算與儲存區分開。它可讓儲存資料更具成本效益、讓您儲存幾乎無限量的資料、並可隨需擴充（或縮減）工作負載、讓資料和租戶重新平衡等管理工作變得更輕鬆。S3相容的儲存系統可利用所有這些功能、在同一個位置將所有事件的資料民主化、免除複雜資料工程的需求。如需瞭解為何應使用卡夫卡階層式儲存設備的詳細資訊、請查看 "本文作者：Conflent"。

為何選擇NetApp StorageGRID 解決方案來進行階層式儲存？

NetApp是領先業界的物件儲存平台。StorageGRID支援業界標準物件API（包括Amazon Simple Storage Service（S3）API）的軟體定義物件式儲存解決方案。StorageGRID可大規模儲存及管理非結構化資料、提供安全且持久的物件儲存。StorageGRID內容會放在適當的位置、適當的時間、以及適當的儲存層、以最佳化工作流程、並降低全球分散式多媒體的成本。

與眾不同之處在於StorageGRID 其資訊生命週期管理（ILM）原則引擎、可實現原則導向的資料生命週期管理。原則引擎可以使用中繼資料來管理整個生命週期內的資料儲存方式、以期一開始就最佳化效能、並自動最佳化資料存留期的成本與持久性。

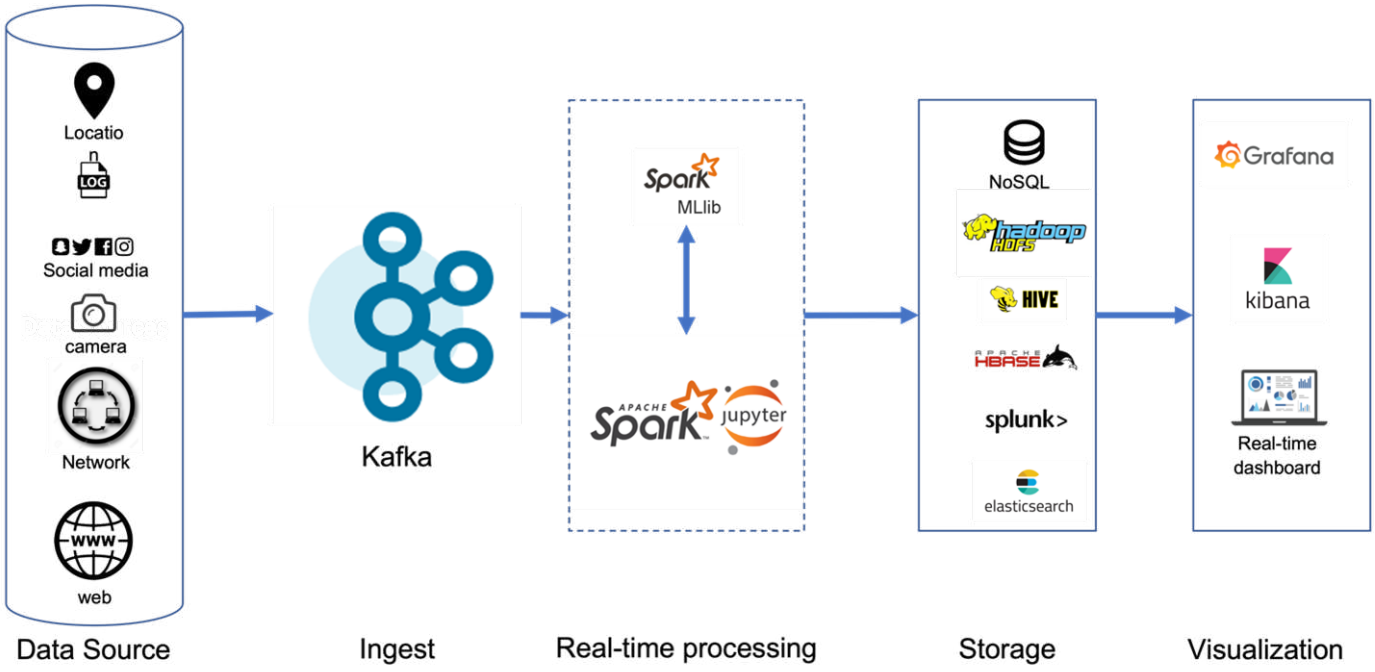
實現一致的分層儲存

階層式儲存的基本概念是將資料儲存的工作與資料處理區分開。如此分離之後、資料儲存層和資料處理層就變得更加容易獨立擴充。

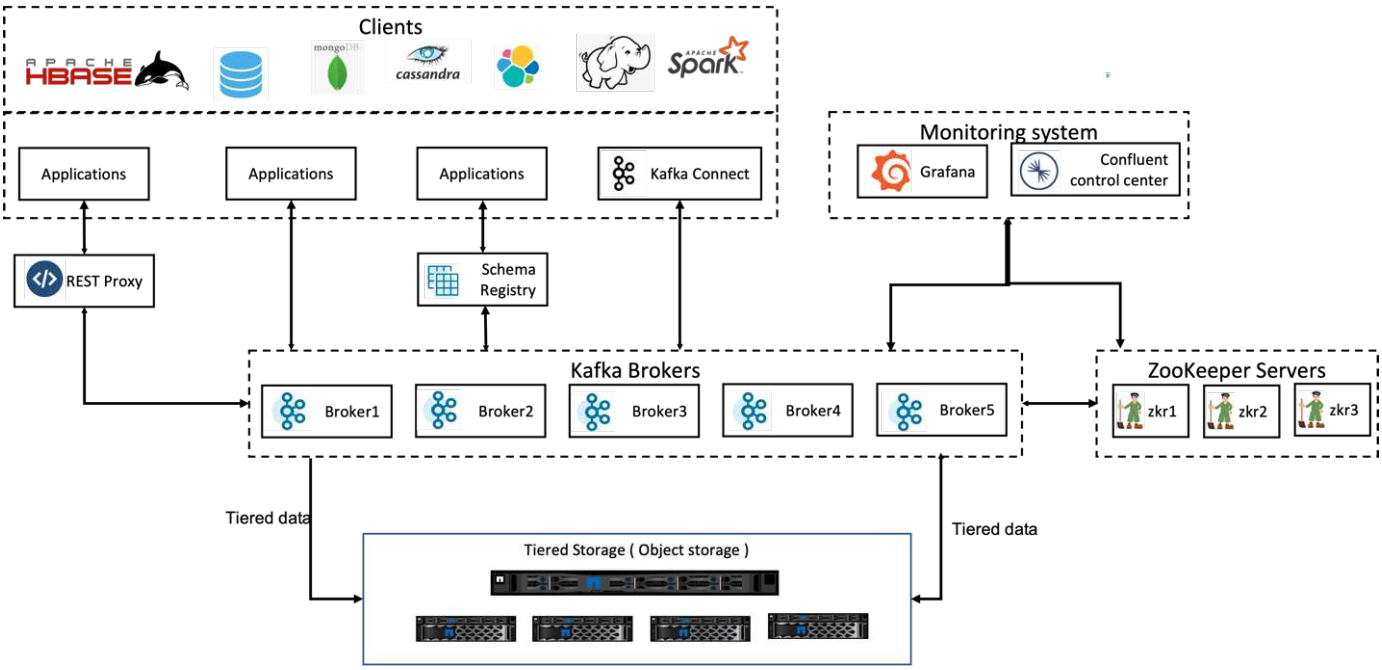
Conflent的階層式儲存解決方案必須面對兩個因素。首先、IT必須解決或避免常見的物件存放區一致性和可用度內容、例如清單作業不一致、偶爾物件無法使用。第二、IT必須正確處理階層式儲存設備與Kafka複寫與容錯模式之間的互動、包括殭屍領導廠商可能繼續分層調整偏移範圍。NetApp物件儲存設備提供一致的物件可用度和HA模式、讓舊舊儲存設備可用於層級偏移範圍。NetApp物件儲存設備提供一致的物件可用度、以及HA模式、可讓舊舊儲存設備用於層級偏移範圍。

透過階層式儲存設備、您可以使用高效能平台、在串流資料的尾端附近進行低延遲的讀取和寫入、也可以使用更

便宜、可擴充的物件存放區（例如NetApp StorageGRID 流通量高的歷史讀取）。我們也提供適用於Spark with NetApp儲存控制器的技術解決方案、詳情請見此處。下圖顯示Kafka如何融入即時分析管道。



下圖說明NetApp StorageGRID 的物件儲存層如何融入ConFluent Kafka的物件儲存層。



"下一步：解決方案架構詳細資料。"

解決方案架構詳細資料

"上一篇：簡介。"

本節涵蓋用於ConFluent驗證的硬體與軟體。此資訊適用於透過NetApp儲存設備進行的ConFluent Platform部

署。下表涵蓋已測試的解決方案架構和基礎元件。

解決方案元件	詳細資料
Connent Kafka 6.2版	• 三位Zookkeepers • 五個代理伺服器 • 五種工具伺服器 • 單一Grafana • 單一控制中心
Linux（Ubuntu 18.04）	所有伺服器
適用於StorageGRID 階層式儲存的NetApp解決方案	• 軟件StorageGRID • 1個SG1000（負載平衡器） • 4個SGF6024 • 4 x 24 x 800 SSD • S3傳輸協定 • 4 x 100GbE（代理StorageGRID 程式與實例之間的網路連線）
15部Fujitsu PRIMERGY RX2540伺服器	每個配備：* 2個CPU、總共16個實體核心* Intel Xeon * 256GB實體記憶體* 100GbE雙埠

"下一步：技術總覽。"

技術總覽

"上一篇：解決方案架構詳細資料。"

本節說明本解決方案所使用的技術。

NetApp StorageGRID

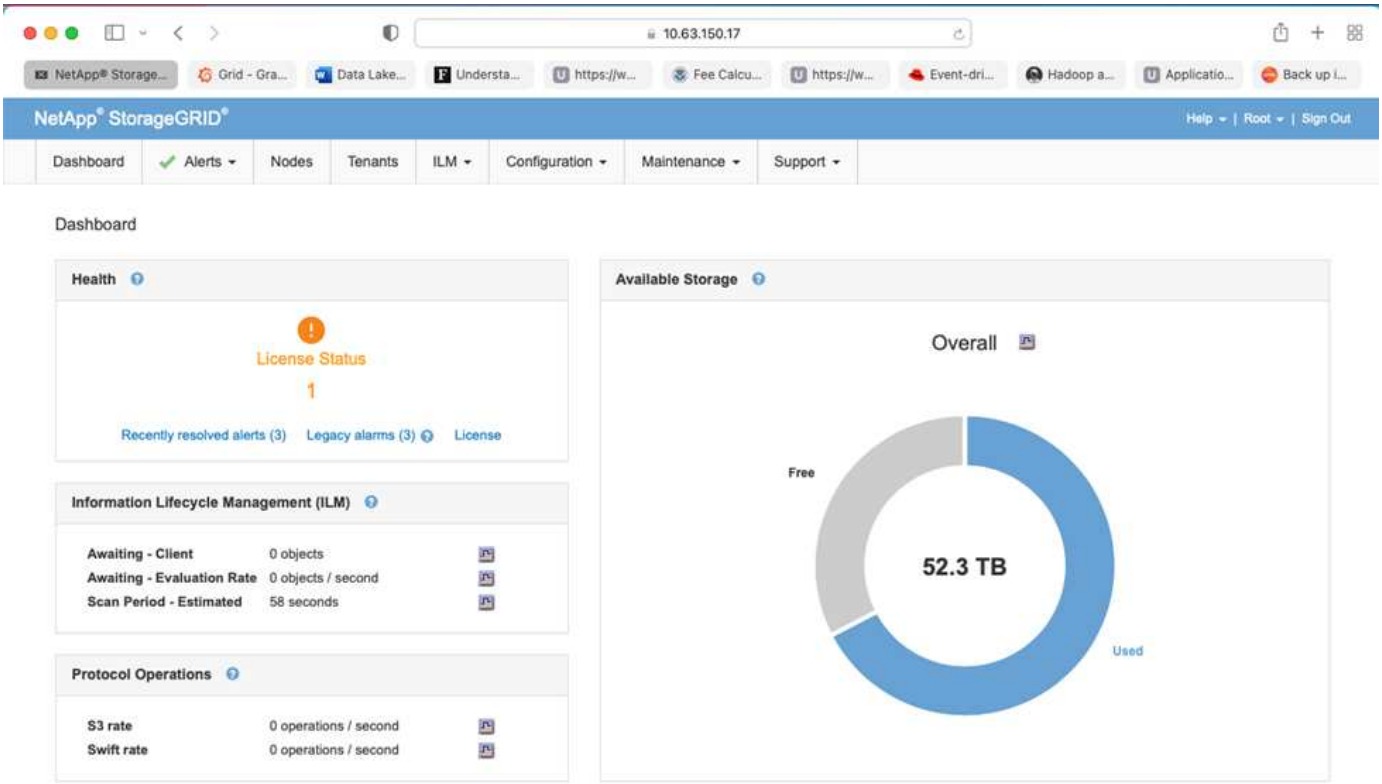
NetApp StorageGRID 產品是高效能且具成本效益的物件儲存平台。透過階層式儲存設備、儲存在本機儲存設備或代理程式SAN儲存設備上的Confluent Kafka上的大部分資料都會卸載到遠端物件存放區。此組態可減少重新平衡、擴充或縮減叢集或更換故障的代理程式所需的時間與成本、進而大幅改善營運。物件儲存在管理位於物件存放區層的資料方面扮演著重要角色、因此挑選適當的物件儲存非常重要。

利用分散式節點型網格架構、提供智慧型原則導向的全球資料管理功能。StorageGRID它透過無所不在的全域物件命名空間、加上精密的資料管理功能、簡化PB非結構化資料和數十億個物件的管理。單一通話物件存取功能可延伸至各個站台、並簡化高可用度架構、同時確保無論站台或基礎架構停機、都能持續存取物件。

多租戶功能可在同一個網格內安全地維護多個非結構化雲端和企業資料應用程式、進而提高NetApp StorageGRID 的ROI和使用案例。您可以利用中繼資料導向的物件生命週期原則來建立多個服務層級、以最佳化跨多個地理區的持久性、保護、效能和位置。使用者可以調整資料管理原則、並監控及套用流量限制、以便在瞬息萬變的IT環境中、隨著需求變化而不中斷地重新調整資料環境。

利用**Grid Manager**輕鬆管理

透過瀏覽器型的圖形介面StorageGRID 、您可以在StorageGRID 單一窗口中、設定、管理及監控分散於全球各地的整個系統。



您可以使用StorageGRID 「資訊網管理程式」介面執行下列工作：

- 管理分散在全球各地、PB規模的物件儲存庫、例如影像、視訊和記錄。
- 監控網格節點和服務、確保物件可用度。
- 使用資訊生命週期管理（ILM）規則、管理物件資料隨時間擺放的位置。這些規則可控制物件擷取後的資料處理方式、資料保護方式、資料儲存位置、以及資料儲存時間。
- 監控系統內的交易、效能和作業。

資訊生命週期管理原則

根據特定的效能和資料保護需求、支援靈活的資料管理原則、包括保留物件的複本複本、以及使用EC（銷毀編碼）配置（例如2+1和4+2）來儲存物件。StorageGRID隨著工作負載和需求隨時間變化、ILM原則也必須隨時間而改變、這是很常見的做法。修改ILM原則是一項核心功能、讓StorageGRID 客戶能夠快速輕鬆地因應瞬息萬變的環境。請檢查 "ILM原則" 和 "ILM規則" 安裝於此。StorageGRID

效能

透過新增更多儲存節點（例如VM、裸機或特定用途的應用裝置）來擴充效能StorageGRID "SG5712、SG5760、SG6060或SGF6024"。在我們的測試中、我們使用SGF6024應用裝置、以最小尺寸的三節點網格、超越Apache Kafka的關鍵效能要求。隨著客戶使用額外的代理商來擴充其Kafka叢集、他們可以新增更多儲存節點來提升效能和容量。

負載平衡器和端點組態

本功能提供Grid Manager UI（使用者介面）和REST API端點、StorageGRID 以檢視、設定及管理StorageGRID 您的不實系統、以及稽核記錄來追蹤系統活動。為了提供高可用度的S3端點給ConFluent Kafka階層式儲存設備、我們實作StorageGRID 了一套以服務形式在管理節點和閘道節點上執行的負載平衡器。此外、負載平衡器也會管理本機流量、並與GSLB（全域伺服器負載平衡）對話、以協助災難恢復。

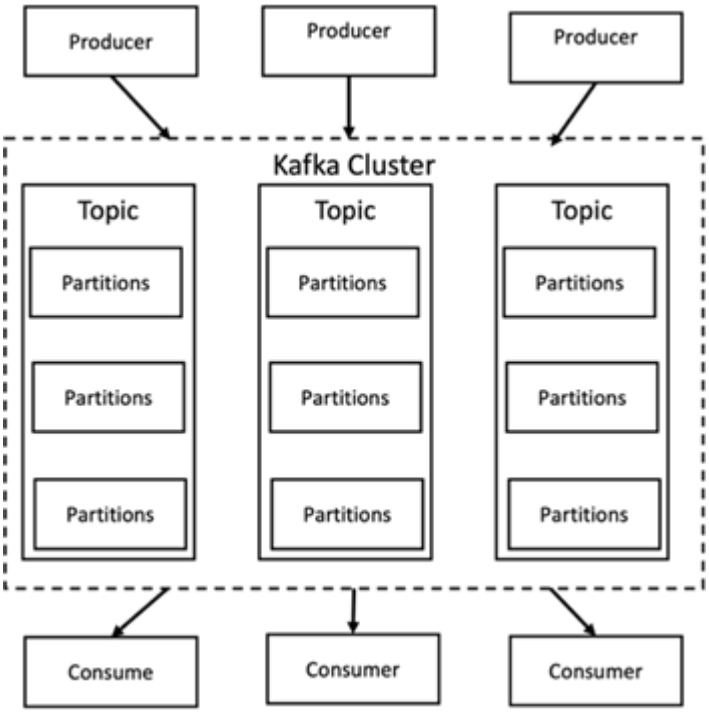
為了進一步強化端點組態、StorageGRID 支援內建於管理節點的流量分類原則、讓您監控工作負載流量、並將各種服務品質（QoS）限制套用至工作負載。流量分類原則會套用至StorageGRID 閘道節點和管理節點的「動態負載平衡器」服務上的端點。這些原則可協助限制流量及監控。

流量分類**StorageGRID**

包含內建QoS功能。StorageGRID流量分類原則可協助監控來自用戶端應用程式的不同類型S3流量。然後您可以建立並套用原則、根據輸入/輸出頻寬、讀取/寫入並行要求數或讀取/寫入要求率來限制此流量。

Apache Kafka

Apache Kafka是以Java和Scala寫入串流處理的軟體匯流排架構實作。它旨在提供統一化、高處理量、低延遲的平台、以處理即時資料饋送。卡夫卡可連線至外部系統、以便透過Kafka Connect匯出及匯入資料、並提供Kafka串流處理程式庫、即Java串流處理程式庫。Kafka使用以TCP為基礎的二進位傳輸協定、針對效率最佳化、並仰賴「訊息集」抽象化、將訊息自然地分組在一起、以降低網路往返的成本。如此一來、就能進行更大的連續磁碟作業、較大的網路封包和鄰近的記憶體區塊、進而讓Kafka將一串爆發性的隨機訊息寫入串流變成線性寫入。下圖說明Apache Kafka的基本資料流程。



Kafka儲存的關鍵價值訊息來自於任意數量的稱為「生產商」的程序。資料可分割成不同主題中的不同分割區。在磁碟分割內、訊息會依照偏移量（訊息在磁碟分割內的位置）嚴格排序、並與時間戳記一起索引及儲存。其他稱為「使用者」的程序也可以從分割區讀取訊息。對於串流處理、Kafka提供的STREAMS API可讓您寫入Java應用程式、以使用Kafka的資料、並將結果寫回Kafka。Apache Kafka也可搭配外部串流處理系統使用、例如Apache Apex、Apache Flink、Apache Spark、Apache Storm及Apache NiFI。

Kafka會在一個或多個伺服器（稱為代理程式）的叢集上執行、所有主題的分割區會分散到叢集節點。此外、分

割區也會複寫到多個代理程式。此架構可讓Kafka以容錯的方式提供大量訊息串流、並讓它取代一些傳統的訊息系統、例如Java Message Service（JMS）、進階訊息佇列傳輸協定（AMQP）等。自0.11.0.0發行以來、Kafka提供交易式寫入功能、使用STREAMS API提供一次完整的串流處理。

卡夫卡支援兩種主題：一般主題和精簡主題。一般主題可設定保留時間或空間限制。如果有記錄超過指定的保留時間、或磁碟分割超出空間限制、則Kafka可以刪除舊資料以釋放儲存空間。根據預設、主題的保留時間設定為7天、但也可以無限期儲存資料。對於精簡主題、記錄不會因時間或空間界限而過期。相反地、Kafka會將稍後的訊息視為舊訊息的更新、並保證不會刪除每個金鑰的最新訊息。使用者可以撰寫所謂的tombstone訊息、並針對特定金鑰使用null值、以完全刪除訊息。

卡夫卡有五大API：

- *監製API。*允許應用程式發佈記錄串流。
- *消費者API。*允許應用程式訂閱主題和處理記錄串流。
- *連接器API.*執行可重複使用的生產商與消費者API、這些API可將主題連結至現有的應用程式。
- *串流API.*此API會將輸入串流轉換成輸出、並產生結果。
- *管理API。*用於管理Kafka主題、代理人及其他Kafka物件。

消費者與製造商API以Kafka訊息傳輸協定為基礎、為Java中的Kafka消費者與製造商用戶端提供參考實作。基礎訊息傳輸協定是一種二進位傳輸協定、開發人員可用來以任何程式設計語言撰寫自己的消費者或生產用戶端。這可讓Kafka從Java虛擬機器（JVM）生態系統中解放。Apache Kafka wikis會維護可用的非Java用戶端清單。

Apache Kafka使用案例

Apache Kafka最受訊息、網站活動追蹤、指標、記錄彙總、串流處理、 事件來源及提交記錄。

- Kafka的處理量、內建分割區、複寫及容錯能力均有提升、因此是大型訊息處理應用程式的理想解決方案。
- Kafka可以在追蹤管道中重建使用者的活動（頁面檢視、搜尋）、做為一組即時發佈訂閱摘要。
- Kafka經常用於營運監控資料。這包括彙總分散式應用程式的統計資料、以產生集中化的作業資料饋送。
- 許多人使用Kafka來取代記錄彙總解決方案。記錄集合通常會從伺服器收集實體記錄檔、並將它們放在中央位置（例如檔案伺服器或HDFS）進行處理。Kafka會將檔案詳細資料擷取出來、並將記錄或事件資料當作訊息串流來提供更簡潔的抽象化。如此可降低延遲處理、更輕鬆支援多個資料來源和分散式資料使用。
- 卡夫卡的許多使用者會處理由多個階段組成的管線資料、其中原始輸入資料會從卡夫卡主題中消耗、然後彙總、豐富或以其他方式轉化為新主題、以供進一步消費或後續處理。例如、推薦新聞文章的處理管道可能會從RSS摘要串流文章內容、然後將其發佈至「文章」主題。進一步處理可能會將此內容正規化或重複資料刪除、並將已清除的文章內容發佈至新主題、最後的處理階段可能會嘗試將此內容推薦給使用者。這類處理管道會根據個別主題、建立即時資料流程的圖表。
- 事件來源是應用程式設計的一種樣式、其狀態變更會記錄為依時間順序排列的記錄順序。卡夫卡支援非常大的儲存記錄資料、因此對於以這種風格建置的應用程式來說、它是絕佳的後端。
- Kafka可以做為分散式系統的外部提交記錄。此記錄有助於在節點之間複寫資料、並可做為故障節點還原資料的重新同步機制。Kafka的記錄壓縮功能有助於支援此使用案例。

Confluent

Conflent Platform是企業級平台、具備進階功能、可協助加速應用程式開發與連線、透過串流處理實現轉型、大規模簡化企業營運、並符合嚴苛的架構要求。Conflent是由Apache Kafka原創原創者所打造、以企業級功能擴展Kafka的優勢、同時免除Kafka管理或監控的負擔。如今、超過80%的財星雜誌100大企業都採用資料串流技術、大部分企業都使用Conflent技術。

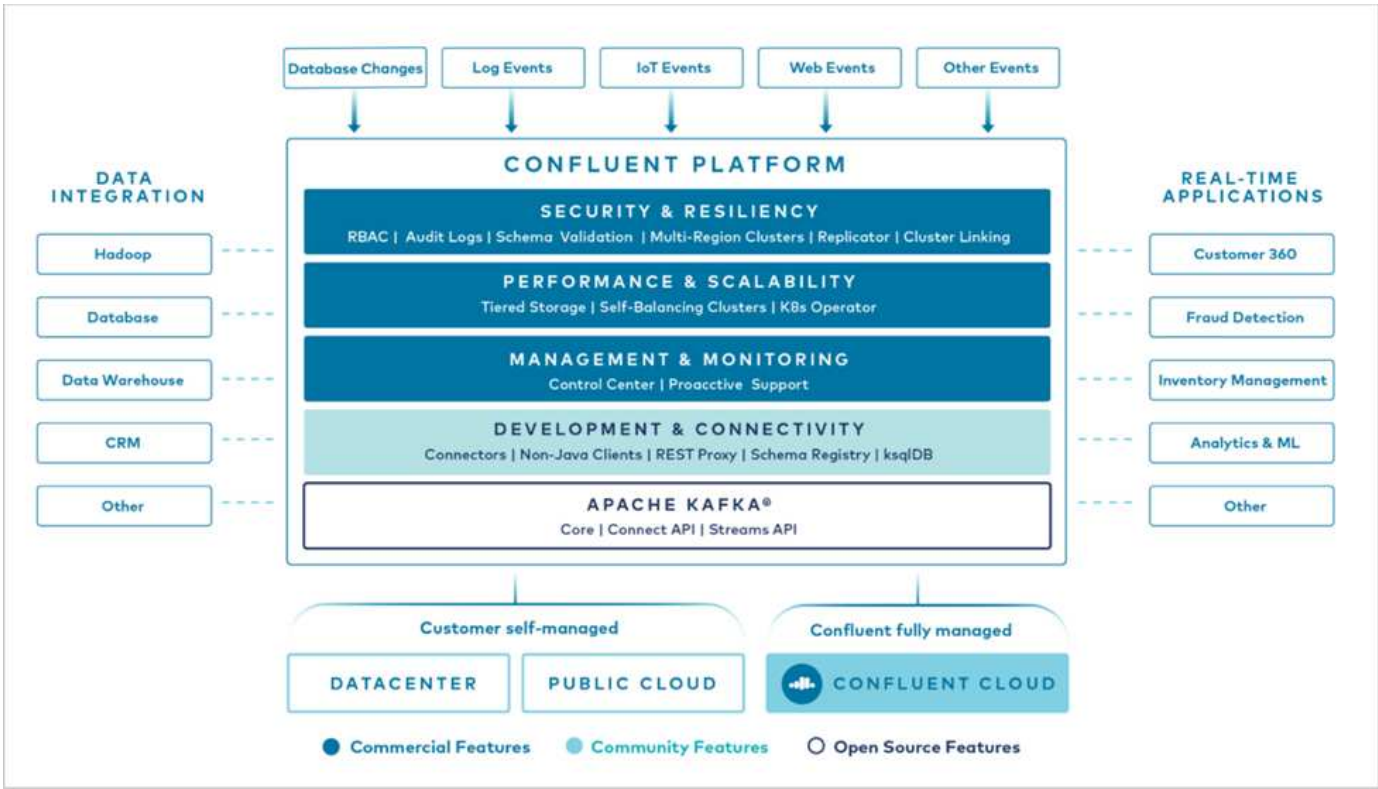
為何選擇**Conflent**？

藉由將歷史與即時資料整合至單一的集中式事實來源、Conflent可讓您輕鬆建置全新類別的現代化事件導向應用程式、取得通用資料管線、並以完整的擴充性、效能與可靠性、釋放強大的新使用案例。

什麼是**ConnFluent**的用途？

Conflent Platform可讓您專注於從資料中獲取商業價值、而非擔心基礎機制、例如資料如何在不同的系統之間傳輸或整合。具體而言、Conflent Platform可簡化資料來源與Kafka之間的連線、建置串流應用程式、以及保護、監控及管理Kafka基礎架構。如今、Conflent Platform可用於金融服務、全通路零售和自主汽車等多種產業的各種使用案例、以及詐欺偵測、 微服務和IoT。

下圖顯示ConnFluent Kafka平台元件。



Connent的事件串流技術總覽

在Conflent Platform的核心是 "Apache Kafka"是最受歡迎的開放原始碼分散式串流平台。卡夫卡的主要功能如下：

- 發佈及訂閱記錄串流。
- 以容錯的方式儲存記錄串流。
- 處理記錄串流。

隨裝即用的Conflent Platform也包括架構登錄、REST Proxy、總共100多個預先建置的Kafka連接器和ksqlDB。

Conflent平台的企業級功能總覽

- * Confluent Control Cent.*一種GUI型系統、用於管理及監控Kafka。它可讓您輕鬆管理Kafka Connect、以

及建立、編輯及管理與其他系統的連線。

- * Kubernetes的Conflent。* Kubernetes的Connent是Kubernetes營運者。Kubernetes營運者提供特定平台應用程式的獨特功能和需求、藉此擴充Kubernetes的協調功能。對於Conflent Platform、這包括大幅簡化Kubernetes上的Kafka部署程序、以及自動化典型的基礎架構生命週期工作。
- *連接至Kafka的Confluent連接器*連接器使用Kafka Connect API將Kafka連接至其他系統、例如資料庫、金鑰值儲存區、搜尋索引和檔案系統。Conflent Hub提供可下載的連接器、適用於最受歡迎的資料來源和接收器、包括這些連接器的完整測試和支援版本、以及Conflent Platform。如需詳細資料、請參閱 "請按這裡"。
- *自我平衡叢集。*提供自動負載平衡、故障偵測及自我修復功能。它支援視需要新增或汰換代理商、無需手動調校。
- * Confluent叢集連結。*直接將叢集連線在一起、並透過連結橋接器將主題從一個叢集鏡射到另一個叢集。叢集連結可簡化多資料中心、多叢集及混合雲部署的設定。
- * Confluent自動資料平衡器。*監控叢集的代理程式數量、分割區大小、分割區數目、以及叢集內的領導者數量。它可讓您將資料移轉至整個叢集、以建立平均工作負載、同時節流重新平衡流量、將對正式作業工作負載的影響降至最低、同時重新平衡。
- * Confluent replicator。*讓您在多個資料中心中維護多個Kafka叢集變得比以往更輕鬆。
- *分層儲存。*提供使用您最喜愛的雲端供應商儲存大量Kafka資料的選項、藉此降低營運負擔和成本。透過階層式儲存設備、您只能在需要更多運算資源時、將資料保存在具成本效益的物件儲存設備上、並擴充代理商。
- * Connent Jms用戶端。* Conflent Platform包含適用於Kafka的與Jms相容的用戶端。此Kafka用戶端實作了JMS 1.1標準API、使用Kafka Brokers做為後端。如果您使用的是使用Jms的舊應用程式、而且想要以Kafka取代現有的Jms訊息代理程式、這項功能就很實用。
- * Conflent MQtT Proxy。*提供一種從MQtT裝置和閘道直接發佈資料至Kafka的方法、而不需要中間的MQtT代理程式。
- * Confluent安全外掛程式。* Confluent安全外掛程式可用來新增各種Confluent Platform工具和產品的安全功能。目前有一個外掛程式可供Conflent REST Proxy使用、可協助驗證傳入要求、並將驗證的主體傳播至向Kafka的要求。這可讓Conflent REST Proxy用戶端利用Kafka代理程式的多租戶安全功能。

"下一步：流暢驗證。"

一致驗證

"上一篇：技術總覽。"

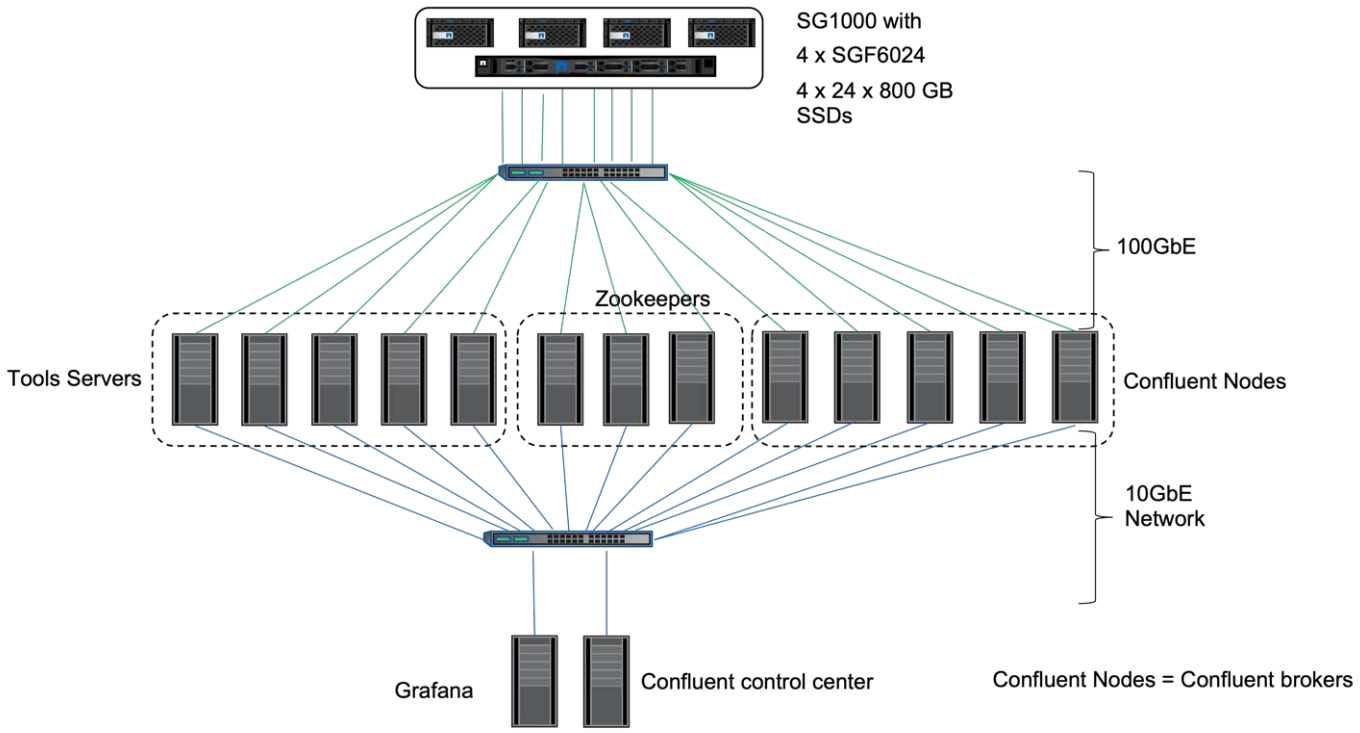
我們在NetApp StorageGRID 的《Consent Platform 6.2多層式儲存設備》中執行驗證。NetApp與Conflent團隊共同進行這項驗證、並執行驗證所需的測試案例。

ConFluent平台設定

我們使用下列設定進行驗證。

為了驗證、我們使用三個zookeepers、五個代理商、五個執行伺服器的測試指令碼、256GB RAM的命名工具伺服器、以及16個CPU。對於NetApp儲存設備、StorageGRID 我們使用的是搭載四個SGF6024s的Sg1000負載平衡器。儲存設備與代理商均透過100GbE連線進行連線。

下圖顯示用於ConFluent驗證的組態網路拓撲。



這些工具伺服器可做為將要求傳送至Confluent節點的應用程式用戶端。

一致的階層式儲存組態

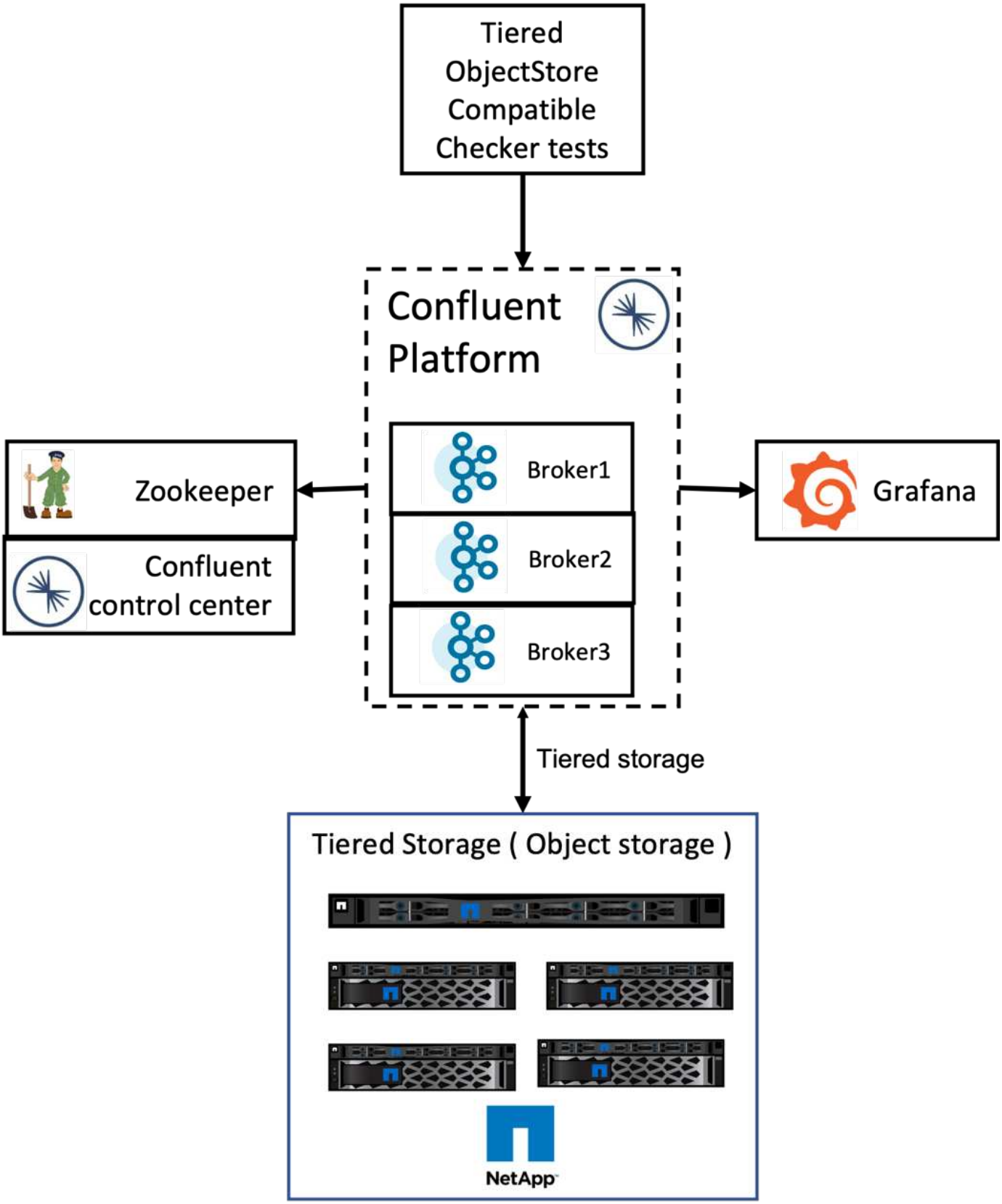
階層式儲存組態需要卡夫卡的下列參數：

```
Confluent.tier.archiver.num.threads=16
confluent.tier.fetcher.num.threads=32
confluent.tier.enable=true
confluent.tier.feature=true
confluent.tier.backend=S3
confluent.tier.s3.bucket=kafkasgdbucket1-2
confluent.tier.s3.region=us-west-2
confluent.tier.s3.cred.file.path=/data/kafka/.ssh/credentials
confluent.tier.s3.aws.endpoint.override=http://kafkasgd.rtpppe.netapp.com:
10444/
confluent.tier.s3.force.path.style.access=true
```

為了驗StorageGRID 證、我們使用了搭配HTTP傳輸協定的功能、但HTTPS也能正常運作。存取金鑰和秘密金鑰會儲存在「confine.Tier .s3.cred.file.path」參數中提供的檔案名稱中。

NetApp物件儲存- StorageGRID 功能

我們將單一站台組態設定為StorageGRID 供驗證之用。



驗證測試

我們完成下列五個驗證測試案例。這些測試是在Trogdor架構上執行。前兩項是功能測試、其餘三項是效能測試。

物件存放區正確性測試

此測試可判斷物件存放區API上的所有基本作業（例如、Get/PUT / DELETE）是否能根據階層式儲存設備的需求順利運作。這是一項基本測試、每個物件存放區服務都應該在下列測試之前通過。這是一項堅定的測試、無論通過或失敗。

分層功能正確性測試

這項測試可判斷端點對端點階層式儲存功能是否與通過或失敗的權證測試搭配運作良好。測試會建立測試主題、預設設定為啟用分層、並大幅減少熱設定大小。它會產生事件串流至新建立的測試主題、等待代理程式將區段歸檔至物件存放區、然後使用事件串流、並驗證耗用的串流是否符合產生的串流。可設定產生給事件串流的訊息數量、讓使用者根據測試需求產生足夠大的工作負載。減少的熱集大小可確保使用者只能從物件存放區取得作用中區段以外的擷取、這有助於測試物件存放區的讀取正確性。我們已執行這項測試、且不需要進行物件存放區故障注入。我們在StorageGRID 其中一個節點停止服務管理程式服務、並驗證端點對端點功能是否可與物件儲存搭配運作、藉此模擬節點故障。

階層擷取基準測試

此測試可驗證階層式物件儲存設備的讀取效能、並在基準測試產生的區段負載過重時、檢查範圍擷取讀取要求。在這個基準測試中、Connent開發了自訂用戶端、以滿足層級擷取要求。

產生消耗的工作負載基準測試

此測試透過區段歸檔、間接在物件存放區上產生寫入工作負載。讀取工作負載（區段讀取）是在使用者群組擷取區段時、從物件儲存區產生的。此工作負載是由測試指令碼產生。此測試會檢查平行執行緒中物件儲存設備的讀取和寫入效能。我們在進行分層功能正確性測試時、測試了是否有物件存放區故障注入。

保留工作負載基準測試

此測試檢查了物件存放區在繁重主題保留工作負載下的刪除效能。保留工作負載是使用測試指令碼產生、此指令碼會產生許多訊息、並與測試主題平行。測試主題是以積極的大小型和時間型保留設定來設定、導致事件串流持續從物件存放區中清除。然後將區段歸檔。這導致代理程式在物件儲存區中刪除大量內容、並收集物件存放區刪除作業的效能。

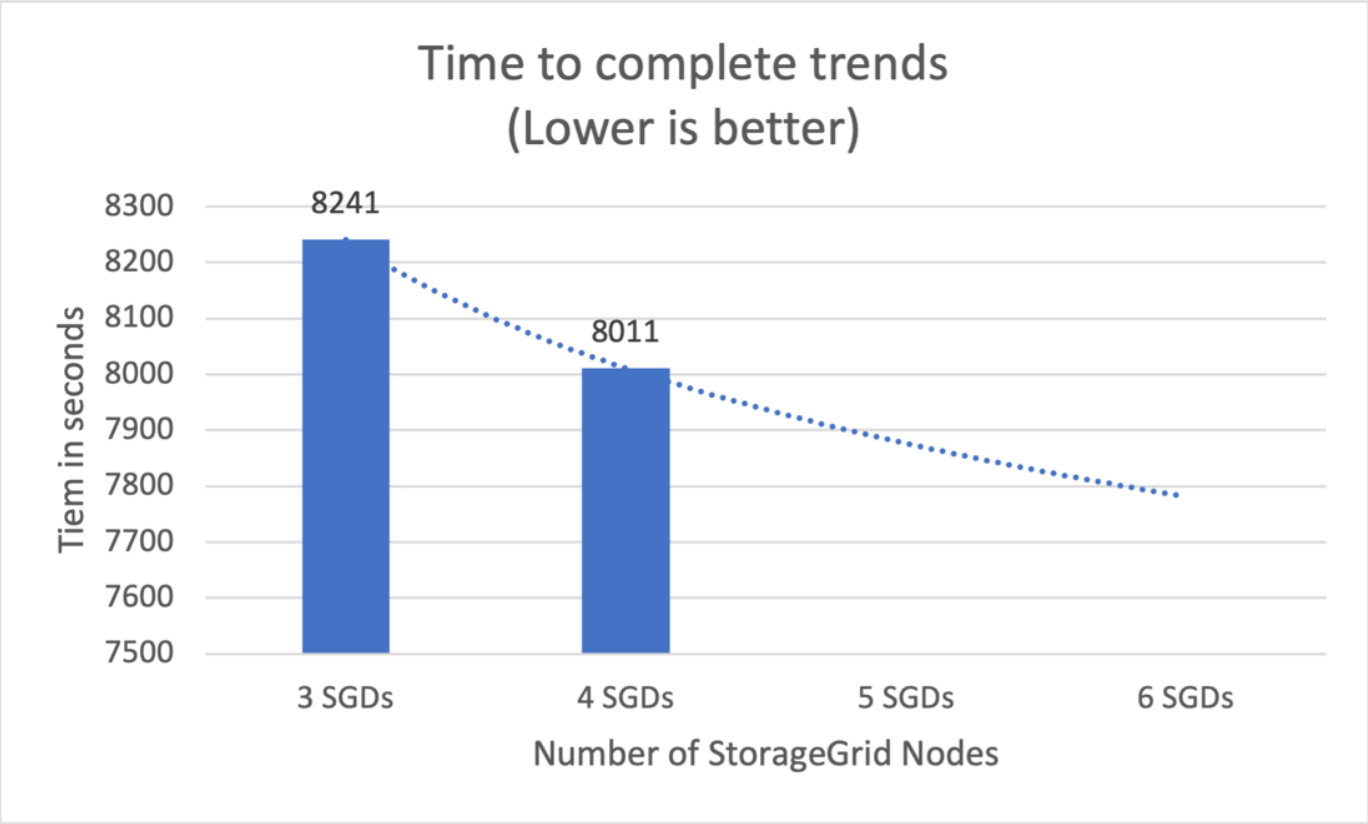
"下一步：效能測試與擴充性。"

可擴充性的效能測試

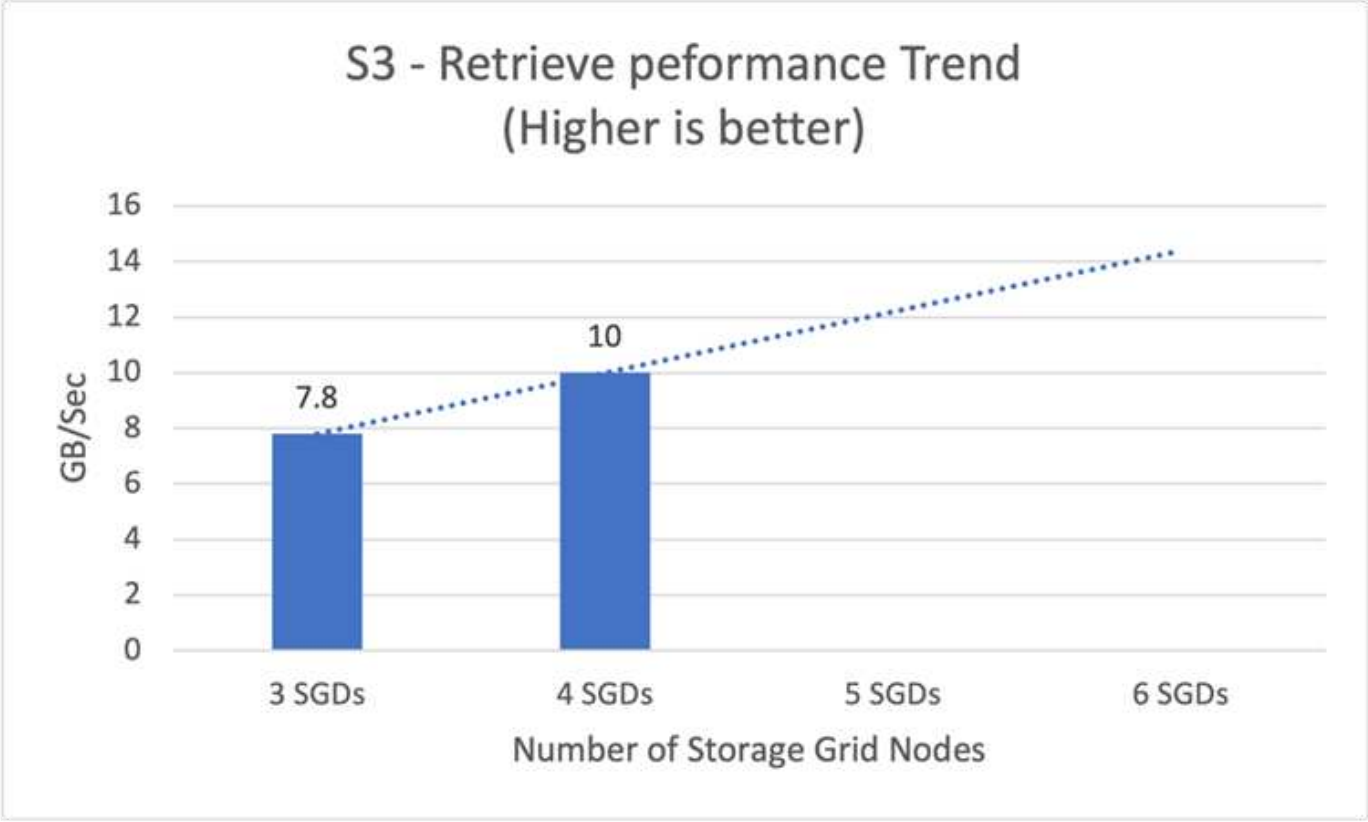
"上一篇：Confluent驗證。"

我們利用NetApp StorageGRID 的支援功能、針對生產與消費型工作負載執行三到四個節點的階層式儲存測試。根據我們的測試、完成時間和效能結果與StorageGRID 各個節點的數量直接成比例。此功能需要至少三個節點。StorageGRID

- 當儲存節點數量增加時、完成產品和消費者作業的時間會線性縮短。



- S3擷取作業的效能會根據StorageGRID 各個節點的數量而線性提升。支援多達200個StorgeGRID節點。StorageGRID



"下一步：Connent S3連接器。"

Connent S3連接器

"先前版本：效能測試搭配擴充性。"

Amazon S3 Sink Connector會以Avro、Json或位元組格式、將Apache Kafka主題的資料匯出至S3物件。Amazon S3 Sink連接器會定期輪詢Kafka的資料、然後再將資料上傳至S3。分區工具用於將每個Kafka分區的資料分割成區塊。每一區塊的資料都會以S3物件表示。金鑰名稱會編碼主題、Kafka分割區、以及此資料區塊的起始偏移。

在此設定中、我們會示範如何使用Kafka S3接收器連接器、直接從Kafka讀取和寫入物件儲存區中的主題。在這項測試中、我們使用獨立的ConFluent叢集、但此設定適用於分散式叢集。

1. 從Conflent網站下載Conflent Kafka。
2. 將套件解壓縮至伺服器上的資料夾。
3. 匯出兩個變數。

```
Export CONFLUENT_HOME=/data/confluent/confluent-6.2.0
export PATH=$PATH:/data/confluent/confluent-6.2.0/bin
```

4. 對於獨立的Conflent Kafka設定、叢集會在「/tmp」中建立一個暫存根資料夾。它也會建立Zookkeeper、Kafka、架構登錄、Connect、ksql-server、 和控制中心資料夾、並從「$Confluent_home'複製各自的組態檔案。請參閱下列範例：

```
root@stlrx2540m1-108:~# ls -ltr /tmp/confluent.406980/
total 28
drwxr-xr-x 4 root root 4096 Oct 29 19:01 zookeeper
drwxr-xr-x 4 root root 4096 Oct 29 19:37 kafka
drwxr-xr-x 4 root root 4096 Oct 29 19:40 schema-registry
drwxr-xr-x 4 root root 4096 Oct 29 19:45 kafka-rest
drwxr-xr-x 4 root root 4096 Oct 29 19:47 connect
drwxr-xr-x 4 root root 4096 Oct 29 19:48 ksql-server
drwxr-xr-x 4 root root 4096 Oct 29 19:53 control-center
root@stlrx2540m1-108:~#
```

5. 設定Zookkeeper。如果您使用預設參數、則不需要變更任何內容。

```
root@stlrx2540m1-108:~# cat
/tmp/confluent.406980/zookeeper/zookeeper.properties  | grep -iv ^#
dataDir=/tmp/confluent.406980/zookeeper/data
clientPort=2181
maxClientCnxns=0
admin.enableServer=false
tickTime=2000
initLimit=5
syncLimit=2
server.179=controlcenter:2888:3888
root@stlrx2540m1-108:~#
```

在上述組態中、我們更新了「伺服器」。xxx屬性。根據預設、您需要三位Zooka主管才能選擇Kafka。

6. 我們在「/tmp/confine.406980/zookeeper / data」中建立一個具有唯一ID的MyID檔案：

```
root@stlrx2540m1-108:~# cat /tmp/confluent.406980/zookeeper/data/myid
179
root@stlrx2540m1-108:~#
```

我們使用最後數量的IP位址作為MyID檔案。我們使用卡夫卡、連線、控制中心、卡夫卡、卡夫卡、 ksql-server和schema-registry組態。

7. 啟動Kafka服務。

```
root@stlrx2540m1-108:/data/confluent/confluent-6.2.0/bin# confluent
local services  start
The local commands are intended for a single-node development
environment only,
NOT for production usage.

Using CONFLUENT_CURRENT: /tmp/confluent.406980
ZooKeeper is [UP]
Kafka is [UP]
Schema Registry is [UP]
Kafka REST is [UP]
Connect is [UP]
ksqlDB Server is [UP]
Control Center is [UP]
root@stlrx2540m1-108:/data/confluent/confluent-6.2.0/bin#
```

每個組態都有一個記錄資料夾、有助於疑難排解問題。在某些情況下、服務需要更多時間才能啟動。請確定所有服務均已啟動且正在執行。

8. 使用「confhube-hub」安裝Kafka Connect。

```
root@stlrx2540m1-108:/data/confluent/confluent-6.2.0/bin# ./confluent-
hub install confluentinc/kafka-connect-s3:latest
The component can be installed in any of the following Confluent
Platform installations:
  1. /data/confluent/confluent-6.2.0 (based on $CONFLUENT_HOME)
  2. /data/confluent/confluent-6.2.0 (where this tool is installed)
Choose one of these to continue the installation (1-2): 1
Do you want to install this into /data/confluent/confluent-
6.2.0/share/confluent-hub-components? (yN) y

Component's license:
Confluent Community License
http://www.confluent.io/confluent-community-license
I agree to the software license agreement (yN) y
Downloading component Kafka Connect S3 10.0.3, provided by Confluent,
Inc. from Confluent Hub and installing into /data/confluent/confluent-
6.2.0/share/confluent-hub-components
Do you want to uninstall existing version 10.0.3? (yN) y
Detected Worker's configs:
  1. Standard: /data/confluent/confluent-6.2.0/etc/kafka/connect-
distributed.properties
  2. Standard: /data/confluent/confluent-6.2.0/etc/kafka/connect-
standalone.properties
  3. Standard: /data/confluent/confluent-6.2.0/etc/schema-
registry/connect-avro-distributed.properties
  4. Standard: /data/confluent/confluent-6.2.0/etc/schema-
registry/connect-avro-standalone.properties
  5. Based on CONFLUENT_CURRENT:
/tmp/confluent.406980/connect/connect.properties
  6. Used by Connect process with PID 15904:
/tmp/confluent.406980/connect/connect.properties
Do you want to update all detected configs? (yN) y
Adding installation directory to plugin path in the following files:
  /data/confluent/confluent-6.2.0/etc/kafka/connect-
distributed.properties
  /data/confluent/confluent-6.2.0/etc/kafka/connect-
standalone.properties
  /data/confluent/confluent-6.2.0/etc/schema-registry/connect-avro-
distributed.properties
  /data/confluent/confluent-6.2.0/etc/schema-registry/connect-avro-
standalone.properties
  /tmp/confluent.406980/connect/connect.properties
  /tmp/confluent.406980/connect/connect.properties
```

```
Completed
root@stlrx2540m1-108:/data/confluent/confluent-6.2.0/bin#
```

您也可以使用「conlume-hub install confluentinc / Kafka-connect：s3：10.0.3」來安裝特定版本。

9. 依預設、「confluentinc、Kafka-connect、S3」安裝於「/data / conflue/conlume-6.2.0/share/conflum-hub-sue-subs/confluentinc、Kafka-Connect、S3」中。
10. 使用全新的「confluentinc - Kafka-connect - S3」來更新外掛程式路徑。

```
root@stlrx2540m1-108:~# cat /data/confluent/confluent-
6.2.0/etc/kafka/connect-distributed.properties | grep plugin.path
#
plugin.path=/usr/local/share/java,/usr/local/share/kafka/plugins,/opt/co
nnectors,
plugin.path=/usr/share/java,/data/zookeeper/confluent/confluent-
6.2.0/share/confluent-hub-components,/data/confluent/confluent-
6.2.0/share/confluent-hub-components,/data/confluent/confluent-
6.2.0/share/confluent-hub-components/confluentinc-kafka-connect-s3
root@stlrx2540m1-108:~#
```

11. 停止Conflent服務並重新啟動。

```
confluent local services  stop
confluent local services  start
root@stlrx2540m1-108:/data/confluent/confluent-6.2.0/bin# confluent
local services  status
The local commands are intended for a single-node development
environment only,
NOT for production usage.

Using CONFLUENT_CURRENT: /tmp/confluent.406980
Connect is [UP]
Control Center is [UP]
Kafka is [UP]
Kafka REST is [UP]
ksqlDB Server is [UP]
Schema Registry is [UP]
ZooKeeper is [UP]
root@stlrx2540m1-108:/data/confluent/confluent-6.2.0/bin#
```

12. 在「/root/.AWS/IDes’檔案中設定存取ID和秘密金鑰。

```
root@stlrx2540m1-108:~# cat /root/.aws/credentials
[default]
aws_access_key_id = xxxxxxxxxxxx
aws_secret_access_key = xxxxxxxxxxxxxxxxxxxxxxxxx
root@stlrx2540m1-108:~#
```

13. 確認鏟斗可到達。

```
root@stlrx2540m4-01:~# aws s3 –endpoint-url
http://kafkasgd.rtpppe.netapp.com:10444 ls kafkasgdbucket1-2
2021-10-29 21:04:18       1388 1
2021-10-29 21:04:20       1388 2
2021-10-29 21:04:22       1388 3
root@stlrx2540m4-01:~#
```

14. 設定S3和Bucket組態的S3接收器內容檔。

```
root@stlrx2540m1-108:~# cat /data/confluent/confluent-
6.2.0/share/confluent-hub-components/confluentinc-kafka-connect-
s3/etc/quickstart-s3.properties | grep -v ^#
name=s3-sink
connector.class=io.confluent.connect.s3.S3SinkConnector
tasks.max=1
topics=s3_testtopic
s3.region=us-west-2
s3.bucket.name=kafkasgdbucket1-2
store.url=http://kafkasgd.rtpppe.netapp.com:10444/
s3.part.size=5242880
flush.size=3
storage.class=io.confluent.connect.s3.storage.S3Storage
format.class=io.confluent.connect.s3.format.avro.AvroFormat
partitioner.class=io.confluent.connect.storage.partitioner.DefaultPartit
ioner
schema.compatibility=NONE
root@stlrx2540m1-108:~#
```

15. 將幾筆記錄匯入S3儲存區。

```
kafka-avro-console-producer --broker-list localhost:9092 --topic
s3_topic \
--property
value.schema='{"type":"record","name":"myrecord","fields":[{"name":"f1",
"type":"string"}]}'
{"f1": "value1"}
{"f1": "value2"}
{"f1": "value3"}
{"f1": "value4"}
{"f1": "value5"}
{"f1": "value6"}
{"f1": "value7"}
{"f1": "value8"}
{"f1": "value9"}
```

16. 裝入S3接收器連接器。

```
root@stlrx2540m1-108:~# confluent local services connect connector load
s3-sink  --config /data/confluent/confluent-6.2.0/share/confluent-hub-
components/confluentinc-kafka-connect-s3/etc/quickstart-s3.properties
The local commands are intended for a single-node development
environment only,
NOT for production usage.
https://docs.confluent.io/current/cli/index.html
{
  "name": "s3-sink",
  "config": {
    "connector.class": "io.confluent.connect.s3.S3SinkConnector",
    "flush.size": "3",
    "format.class": "io.confluent.connect.s3.format.avro.AvroFormat",
    "partitioner.class":
"io.confluent.connect.storage.partitioner.DefaultPartitioner",
    "s3.bucket.name": "kafkasgdbucket1-2",
    "s3.part.size": "5242880",
    "s3.region": "us-west-2",
    "schema.compatibility": "NONE",
    "storage.class": "io.confluent.connect.s3.storage.S3Storage",
    "store.url": "http://kafkasgd.rtpppe.netapp.com:10444/",
    "tasks.max": "1",
    "topics": "s3_testtopic",
    "name": "s3-sink"
  },
  "tasks": [],
  "type": "sink"
}
root@stlrx2540m1-108:~#
```

17. 檢查S3接收器狀態。

```
root@stlrx2540m1-108:~# confluent local services connect connector
status s3-sink
The local commands are intended for a single-node development
environment only,
NOT for production usage.
https://docs.confluent.io/current/cli/index.html
{
  "name": "s3-sink",
  "connector": {
    "state": "RUNNING",
    "worker_id": "10.63.150.185:8083"
  },
  "tasks": [
    {
      "id": 0,
      "state": "RUNNING",
      "worker_id": "10.63.150.185:8083"
    }
  ],
  "type": "sink"
}
root@stlrx2540m1-108:~#
```

18. 檢查記錄、確定S3接收器已準備好接受主題。

```
root@stlrx2540m1-108:~# confluent local services connect log
```

19. 請查看卡夫卡的主題。

```
kafka-topics --list --bootstrap-server localhost:9092
…
connect-configs
connect-offsets
connect-statuses
default_ksql_processing_log
s3_testtopic
s3_topic
s3_topic_new
root@stlrx2540m1-108:~#
```

20. 檢查S3儲存區中的物件。

```
root@stlrx2540m1-108:~# aws s3 --endpoint-url
http://kafkasgd.rtpppe.netapp.com:10444 ls --recursive kafkasgdbucket1-
2/topics/
2021-10-29 21:24:00        213
topics/s3_testtopic/partition=0/s3_testtopic+0+0000000000.avro
2021-10-29 21:24:00        213
topics/s3_testtopic/partition=0/s3_testtopic+0+0000000003.avro
2021-10-29 21:24:00        213
topics/s3_testtopic/partition=0/s3_testtopic+0+0000000006.avro
2021-10-29 21:24:08        213
topics/s3_testtopic/partition=0/s3_testtopic+0+0000000009.avro
2021-10-29 21:24:08        213
topics/s3_testtopic/partition=0/s3_testtopic+0+0000000012.avro
2021-10-29 21:24:09        213
topics/s3_testtopic/partition=0/s3_testtopic+0+0000000015.avro
root@stlrx2540m1-108:~#
```

21. 若要驗證內容、請執行下列命令、將每個檔案從S3複製到您的本機檔案系統：

```
root@stlrx2540m1-108:~# aws s3 --endpoint-url
http://kafkasgd.rtpppe.netapp.com:10444 cp s3://kafkasgdbucket1-
2/topics/s3_testtopic/partition=0/s3_testtopic+0+0000000000.avro
tes.avro
download: s3://kafkasgdbucket1-
2/topics/s3_testtopic/partition=0/s3_testtopic+0+0000000000.avro to
./tes.avro
root@stlrx2540m1-108:~#
```

22. 若要列印記錄、請使用avro-tools-1.11.0.1.jar（可在 "Apache歸檔"）。

```
root@stlrx2540m1-108:~# java -jar /usr/src/avro-tools-1.11.0.1.jar
tojson tes.avro
21/10/30 00:20:24 WARN util.NativeCodeLoader: Unable to load native-
hadoop library for your platform... using builtin-java classes where
applicable
{"f1":"value1"}
{"f1":"value2"}
{"f1":"value3"}
root@stlrx2540m1-108:~#
```

"下一步：Conflent自我重新平衡叢集。"

ConFluent自我平衡叢集

"先前版本：Kafka S3連接器。"

如果您之前曾管理過Kafka叢集、您可能會熟悉手動將分割區重新指派給不同的代理人所帶來的挑戰、以確保整個叢集的工作負載平衡。對於部署較大Kafka的組織而言、重新安排大量資料可能會令人望而生畏、繁瑣且風險高、尤其是當任務關鍵型應用程式建置於叢集之上時。然而、即使是卡夫卡最小的使用案例、這項程序仍耗時且容易發生人為錯誤。

在實驗室中、我們測試了Confluent自我平衡叢集功能、此功能可根據叢集拓撲變更或負載不均而自動重新平衡。當節點故障或擴充節點需要跨代理程式重新平衡資料時、「Confluent reBalance」測試有助於測量新增代理程式的時間。在傳統的Kafka組態中、隨著叢集成長、要重新平衡的資料量會隨之增加、但在階層式儲存設備中、重新平衡的資料量僅限於少量資料。根據我們的驗證結果、在傳統的Kafka架構中重新平衡階層式儲存設備需要數秒鐘或數分鐘、並隨著叢集成長線性成長。

在自我平衡叢集中、磁碟分割重新平衡已完全自動化、可最佳化Kafka的處理量、加速代理程式擴充、並減輕執行大型叢集的作業負擔。在穩定狀態下、自我平衡叢集會監控所有代理程式的資料偏移、並持續重新分配分割區、以最佳化叢集效能。當垂直或向下擴充平台時、自我平衡叢集會自動辨識新的代理人是否存在、或移除舊的代理人、並觸發後續的分割區重新指派。這可讓您輕鬆新增及取消委任代理人、讓Kafka叢集更具彈性。這些效益不需要手動介入、複雜的數學運算、或是分割區重新指派通常會造成人為錯誤的風險。如此一來、資料重新平衡的完成時間就會大幅縮短、您可以專注於價值更高的事件串流專案、而不需要持續監督叢集。

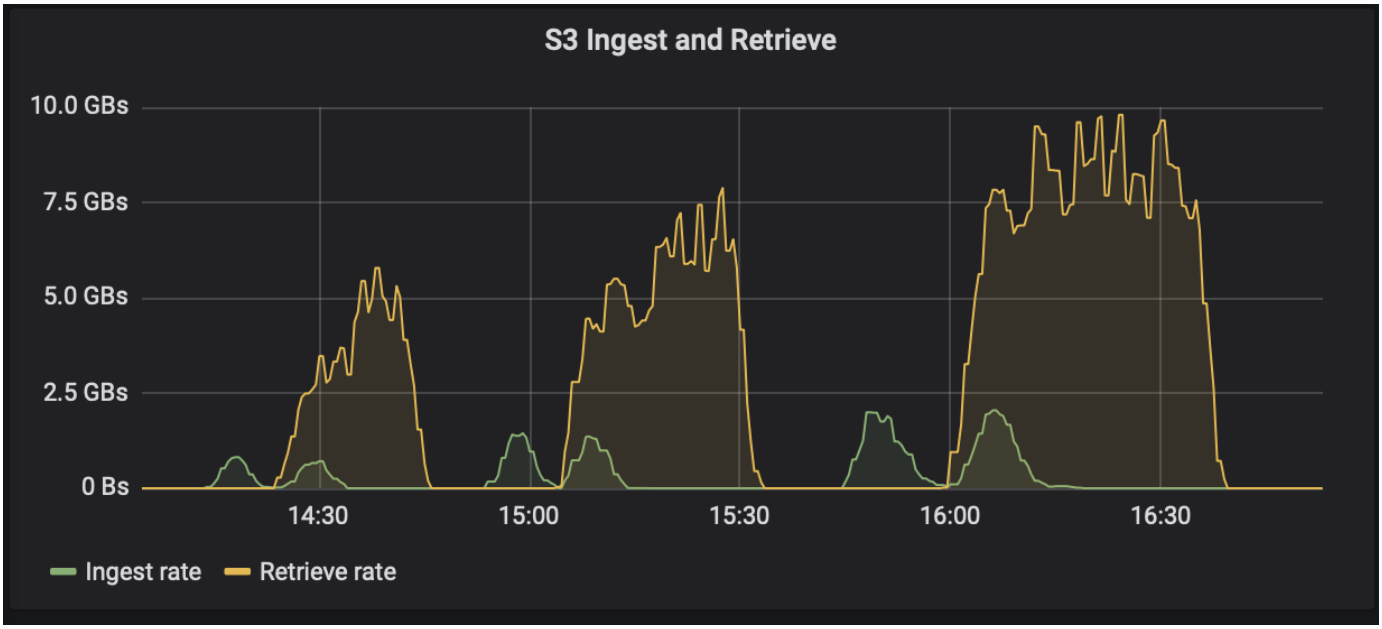
"下一步：最佳實務準則。"

最佳實務準則

"先前版本：Conflent自我重新平衡叢集。"

- 根據我們的驗證結果、S3物件儲存設備最適合Connent來保存資料。
- 我們可以使用高處理量SAN（特別是FC）來保留代理程式的熱資料或本機磁碟、因為在Conflent階層式儲存組態中、 代理資料目錄中的資料大小取決於資料移至物件儲存設備時的區段大小和保留時間。
- 當sege.bytes較高時、物件存放區可提供較佳的效能；我們測試了512MB。
- 在Kafka中、針對主題所產生的每筆記錄、其金鑰或值長度（以位元組為單位）是由「length、key、Value」（長度、金鑰、值）參數所控制。針對VMware、S3物件擷取和擷取效能提升至更高的價值。StorageGRID例如、512位元組提供5.8GBps擷取、而1024位元組提供7.5GBps S3擷取、以及提供接近10Gbps的2048位元組。

下圖顯示根據「length、key、Value」擷取和擷取S3物件。



- * Kafka調校。*若要提升階層式儲存設備的效能、您可以增加TierFetcherNumThreades和TierArchiverNum執行 緒。一般而言、您想要增加TierFetcherNum執行 緒、以符合實體CPU核心數量、並將TierArchiverNum執行 緒增加到CPU核心數量的一半。例如、在伺服器內容中、如果您的機器有八個實體核心、請將confucere.Tier .etcher。num.thread = 8和puceter.Tier。archiver.nm.thread = 4。
- *主題刪除的時間間隔。*刪除主題時、不會立即開始刪除物件儲存區中的記錄區段檔案。相反地、在刪除這些檔案之前、會有一個預設值為3小時的時間間隔。您可以修改組態confluent.tier.topic.delete.check.interval.ms來變更此時間間隔的值。如果您刪除主題或叢集、也可以手動刪除個別儲存區中的物件。
- *階層式儲存內部主題的ACL。*內部部署的建議最佳實務做法是讓ACL授權者能夠處理階層式儲存所使用的內部主題。設定ACL規則、僅限代理使用者存取此資料。如此可保護內部主題、防止未獲授權存取階層式儲存資料和中繼資料。

```
kafka-acls --bootstrap-server localhost:9092 --command-config adminclient-
configs.conf \
--add --allow-principal User:<kafka> --operation All --topic "_confluent-
tier-state"
```

> 將使用者「<Kafka>'」取代為部署中的實際代理主體。

例如、命令「conflexe-層 狀態」會針對階層式儲存設備的內部主題設定ACL。目前只有一個與階層式儲存有關的內部主題。此範例會建立ACL、為內部主題的所有作業提供主要的Kafka權限。

"下一步：規模調整。"

規模調整

"上一篇：最佳實務準則。"

卡夫卡的規模調整可透過四種組態模式執行：簡單、精細、反轉和分割區。

簡單易用

簡易模式適用於首次使用Apache Kafka的使用者或早期狀態使用案例。在此模式下、您需要提供處理量MBps、讀取扇出、保留和資源使用率百分比（預設為60%）等需求。您也可以進入內部部署（裸機、VMware、Kubernetes或OpenStack）或雲端等環境。根據這項資訊、Kafka叢集的規模可提供代理程式、zookeeper、Apache Kafka Connect工作人員、架構登錄、REST Proxy、ksqlDB及Confluent控制中心所需的伺服器數量。

對於階層式儲存設備、請考慮使用精細的組態模式來調整Kafka叢集的規模。精細模式適用於經驗豐富的Apache Kafka使用者或定義明確的使用案例。本節說明生產商、串流處理器及使用者的規模調整。

製造商

若要說明Apache Kafka的生產商（例如原生用戶端、REST Proxy或Kafka連接器）、請提供下列資訊：

- 名稱 Spark.
- *監製者類型。*應用程式或服務、Proxy（REST、MQTT、其他）、以及現有資料庫（RDBMS、NoSQL、其他）。您也可以選取「我不知道」。
- *平均處理量。*以每秒事件為單位（例如1、000、000）。
- *尖峰處理量。*每秒事件數（例如400萬）。
- 平均訊息大小（以位元組為單位）、未壓縮（最多1MB；例如1000）。
- *訊息格式。*選項包括Avro、Json、傳輸協定緩衝區、二進位、文字、 「我不知道」和其他。
- *複寫係數*選項為1、2、3（一致推薦）、4、5、 或6.
- *保留時間*一天（例如）。您想要將資料儲存在Apache Kafka多長時間？在無限時間內輸入-1及任何單位。此計算機假設保留時間為10年、以供無限期保留。
- 選取「啟用分層儲存以減少代理計數並允許無限儲存？」核取方塊。
- 啟用階層式儲存設備時、保留欄位會控制儲存在本機代理程式上的常用資料集。歸檔保留欄位可控制資料儲存在歸檔物件儲存區的時間長度。
- *歸檔儲存保留*一年（例如）。您希望將資料儲存在歸檔儲存設備中多久？在無限期間內輸入-1及任何單位。此計算機假設保留10年、以便無限期保留。
- 成長倍率 1（例如）。如果此參數的值是根據目前處理量而定、請將其設為1。若要根據額外成長來調整規模、請將此參數設為成長倍頻。
- 產生者執行個體數。 10（例如）。將執行多少個生產商執行個體？此輸入是將CPU負載併入規模計算所需的。空白值表示CPU負載未納入計算。

根據此範例輸入、規模調整對生產商有下列影響：

- 未壓縮位元組的平均處理量：1Gbps。未壓縮位元組的尖峰處理量：4Gbps。以壓縮位元組為單位的平均處理量：400Mbps。尖峰處理量（壓縮位元組）：1.6GBps。這是根據預設的60%壓縮率（您可以變更此值）。
    - 所需的總代理熱集儲存容量：31、104 TB、包括複寫、壓縮。所需的非代理歸檔儲存設備總數：378、432TB、已壓縮。使用 "https://fusion.netapp.com" 以利規模調整。StorageGRID

串流處理器必須說明其應用程式或服務、這些應用程式或服務會耗用Apache Kafka的資料、並將資料傳回Apache Kafka。在大多數情況下、這些都是以ksqlDB或Kafka串流建置而成。

- 名稱 Spark streamer。
- *處理時間。*此處理器處理單一訊息需要多久時間？
  - 1毫秒（簡單、無狀態轉換）[範例]、10毫秒（記憶體內有狀態作業）。
  - 100ms（狀態網路或磁碟作業）、1000ms（第三方REST通話）。
  - 我已對此參數進行基準測試、並確切瞭解所需時間。
- 輸出保留。 1天（範例）。串流處理器會將輸出傳回Apache Kafka。您希望此輸出資料在Apache Kafka中儲存多久？在無限期間內輸入-1及任何單位。
- 勾選「啟用分層儲存以減少Broker Count並允許無限儲存？」核取方塊。
- 歸檔儲存保留 1年（例如）。您希望將資料儲存在歸檔儲存設備中多久？在無限期間內輸入-1及任何單位。此計算機假設保留10年、以便無限期保留。
- 輸出傳遞百分比。 100（例如）。串流處理器會將輸出傳回Apache Kafka。傳入處理量的多少百分比會輸出回Apache Kafka？例如、如果傳入處理量為20Mbps、而此值為10、則輸出處理量將為2Mbps。
- 從哪些應用程式讀取？選取「Spark」，這是在生產廠商類型規模調整中使用的名稱。根據上述輸入、您可以預期調整大小對串流處理程序執行個體和主題分割區預估的影響如下：
- 此串流處理器應用程式需要下列數目的執行個體。傳入的主題也可能需要這麼多分割區。請聯絡Conflent以確認此參數。
  - 平均處理量為1、000、沒有成長倍頻
  - 4、000個尖峰處理量、無成長倍頻
  - 平均處理量1、000個、使用成長倍頻
  - 4、000個尖峰處理量、使用成長倍頻

消費者

請描述您的應用程式或服務、這些應用程式或服務會耗用Apache Kafka的資料、而不會產生回Apache Kafka；例如、原生用戶端或Kafka Connector。

- 名稱 Spark消費者。
- *處理時間。*此消費者處理單一訊息所需的時間？
  - 1毫秒（例如、記錄等簡單且無狀態的工作）
  - 10ms（快速寫入資料存放區）
  - 100毫秒（緩慢寫入資料存放區）
  - 1000毫秒（第三方REST通話）
  - 已知持續時間的其他基準程序。
- *使用者類型。*應用程式、Proxy或接收到現有的資料存放區（RDBMS、NoSQL、其他）。
- 從哪些應用程式讀取？將此參數與先前決定的監製者和串流規模連結起來。

根據上述輸入、您必須決定使用者執行個體和主題分割預估的規模。使用者應用程式需要下列執行個體數目。

- 平均處理量為2、000、沒有成長倍頻
- 8000個尖峰處理量、無成長倍頻

- 平均處理量為2、000、包括成長倍率
- 8000個尖峰處理量、包括成長倍頻

傳入的主題也可能需要這個數目的分割區。請聯絡Conflent以確認。

除了對生產商、串流處理器和消費者的要求之外、您還必須提供下列額外要求：

- *重建時間*例如4小時。如果Apache Kafka Broker主機故障、資料遺失、而且配置了新主機來更換故障主機、那麼這台新主機重建的速度必須有多快？如果值不明、請將此參數留白。
- *資源使用率目標（百分比）。*例如60。您希望主機在平均處理量期間的使用率為何？除非您使用Conflent自我平衡叢集、否則Conflent建議使用率為60%、在此情況下、使用率可能會較高。

描述您的環境

- 您的叢集將在哪些環境中執行？ Amazon Web Services、Microsoft Azure、Google雲端平台、內部部署裸機、內部部署VMware、 內部部署OpenStack或內部部署Kubernates？
- *主機詳細資料。*核心數：48（例如）、網路卡類型（10GbE、40GbE、16GbE、1GbE或其他類型）。
- *儲存磁碟區。*主機：12（例如）。每個主機支援多少個硬碟機或SSD？Conflent建議每個主機使用12個硬碟機。
- 儲存容量/磁碟區（單位：**GB**）。 1000（例如）。單一磁碟區可儲存多少GB儲存空間？Connent建議使用1TB磁碟。
- *儲存組態。*如何設定儲存磁碟區？Conflent建議使用RAID10、以充分發揮所有Conflent功能的優勢。JBOD、SAN、RAID 1、RAID 0、RAID 5、 也支援其他類型。
- 單一**Volume**處理量（**Mbps**）。 125（例如）。單一儲存磁碟區每秒讀取或寫入MB的速度有多快？Confluent建議使用標準硬碟、通常處理量為125MBps。
- 記憶體容量（**GB**）。 64（例如）。

確定環境變數之後、請選取「調整叢集大小」。根據上述範例參數、我們針對Connent Kafka決定了下列規模：

- * Apache Kafka.* Broker數量：22。您的叢集需要儲存設備。考慮啟用階層式儲存設備、以減少主機數量、並允許無限儲存。
- * Apache Zookeeper。*計數：5；Apache Kafka Connect工作人員：數：2；架構登錄：數：2；REST Proxy：數：2；ksqlDB：數：2；ConFluent Control Center：數：1。

不需考量使用案例、即可針對平台團隊使用反轉模式。使用分割模式來計算單一主題所需的分割區數量。請參閱 https://eventsizer.io 以反轉和分割模式為基礎進行規模調整。

"下一步：結論。"

結論

"上一篇：規模調整。"

本文件提供將ConFluent階層式儲存設備與NetApp儲存設備搭配使用的最佳實務準則、包括驗證測試、階層式儲存效能結果、調校、ConFluent S3連接器、以及自我平衡功能。考慮到ILM原則、透過多項驗證效能測試的一致效能、以及業界標準的S3 API、NetApp StorageGRID 物件儲存設備是Confluent階層式儲存設備的最佳選擇。

何處可找到其他資訊

若要深入瞭解本文所述資訊、請檢閱下列文件和 / 或網站：

- 什麼是Apache Kafka

  "https://www.confluent.io/what-is-apache-kafka/"

- NetApp 產品文件

  "https://www.netapp.com/support-and-training/documentation/"

- S3-sink參數詳細資料

  "https://docs.confluent.io/kafka-connect-s3-sink/current/configuration_options.html#s3-configuration-options"

- Apache Kafka

  "https://en.wikipedia.org/wiki/Apache_Kafka"

- 在Confluent平台上實現無限儲存

  "https://www.confluent.io/blog/infinite-kafka-storage-in-confluent-platform/"

- 一致的分層儲存設備：最佳實務做法與規模調整

  "https://docs.confluent.io/platform/current/kafka/tiered-storage.html#best-practices-and-recommendations"

- 適用於ConFluent Platform的Amazon S3接收器連接器

  "https://docs.confluent.io/kafka-connect-s3-sink/current/overview.html"

- Kafka尺寸

  "https://eventsizer.io"

- 規模調整StorageGRID

  "https://fusion.netapp.com/"

- Kafka使用案例

  "https://kafka.apache.org/uses"

- 在conf最 合體平台6.0中實現卡夫卡叢集的自我平衡

  "https://www.confluent.io/blog/self-balancing-kafka-clusters-in-confluent-platform-6-0/"

  "https://www.confluent.io/blog/confluent-platform-6-0-delivers-the-most-powerful-event-streaming-platform-to-date/"

版本歷程記錄

版本	日期	文件版本歷程記錄
1.0版	2021年12月	初始版本。

版權資訊

Copyright©2022 NetApp、Inc.版權所有。美國印製本文件中版權所涵蓋的任何部分、不得以任何形式或任何方式（包括影印、錄製、 在未事先取得版權擁有者書面許可的情況下、在電子擷取系統中進行錄音或儲存。

衍生自受版權保護之NetApp資料的軟體必須遵守下列授權與免責聲明：

本軟體係由NetApp「依現狀」提供、不含任何明示或暗示的保證、包括但不限於適售性及特定用途適用性的暗示保證、特此聲明。在任何情況下、NetApp均不對任何直接、間接、偶發、特殊、示範、 或衍生性損害（包括但不限於採購替代商品或服務；使用損失、資料或利潤損失； 或業務中斷）、無論是在合約、嚴格責任或侵權行為（包括疏忽或其他）中、無論是因使用本軟體而產生的任何責任理論（包括疏忽或其他）、即使已被告知可能造成此類損害。

NetApp保留隨時變更本文所述之任何產品的權利、恕不另行通知。除非NetApp以書面明確同意、否則NetApp不承擔因使用本文所述產品而產生的任何責任或責任。使用或購買本產品並不代表NetApp擁有任何專利權利、商標權利或任何其他智慧財產權。

本手冊所述產品可能受到一或多個美國國家/地區的保護專利、國外專利或申請中。

限制權利圖例：政府使用、複製或揭露受DFARS 252.277-7103（1988年10月）和FAR 52-227-19（1987年6月）技術資料與電腦軟體權利條款（c）（1）（ii）分段所述限制。

商標資訊

NetApp、NetApp標誌及所列的標章 http://www.netapp.com/TM 為NetApp、Inc.的商標。其他公司和產品名稱可能為其各自所有者的商標。